Detection and Defense of Cache Pollution Attacks Using Clustering in Named Data Networks

Lin Yao, Zhenzhen Fan, Jing Deng, *Fellow, IEEE*, Xin Fan, *Senior Member, IEEE*, and Guowei Wu

Abstract—Named Data Network (NDN), as a promising information-centric networking architecture, is expected to support next-generation of large-scale content distribution with open in-network cachings. However, such open in-network caches are vulnerable against Cache Pollution Attacks (CPAs) with the goal of filling cache storage with non-popular contents. The detection and defense against such attacks are especially difficult because of CPA's similarities with normal fluctuations of content requests. In this work, we use a clustering technique to detect and defend against CPAs. By clustering the content interests, our scheme is able to distinguish whether they have followed the Zipf-like distribution or not for accurate detections. Once any attack is detected, an attack table will be updated to record the abnormal requests. While such requests are still forwarded, the corresponding content chunks are not cached. Extensive simulations in ndnSIM demonstrate that our scheme can resist CPA effectively with higher cache hit, higher detecting ratio, lower hop count, and lower algorithm complexity compared to other state-of-the-art schemes.

Index Terms—Cache pollution attack, clustering, named data networks

1 Introduction

TCP/IP was designed to provide communications for end-to-end connections. With the explosive growth of networks and networked contents, traditional end-to-end network architecture is overwhelmed. Instead, most communications focus on contents but not the actual carriers of the contents [1], [2], [3]. Named Data Network (NDN), as a promising information-centric networking architecture, focuses on content-centric transmission where a user retrieves a given content directly using the name without caring about the identity or IP address of the content provider. Instead of routing packets based on their destination IP addresses, an NDN router forwards each incoming packet based on its content information.

- *L. Yao is with the Key Laboratory for Ubiquitous Network and Service Software of Liaoning Province, and also with the DUT-RU International School of Information Science & Engineering, Dalian University of Technology, Dalian 116600, China. E-mail: yaolin@dlut.edu.cn.*
- *Z. Fan is with the Key Laboratory for Ubiquitous Network and Service Software of Liaoning Province, and also with the School of Software, Dalian University of Technology, Dalian 116600, China. E-mail: fzz123hd@mail.dlut.edu.cn.*
- *J. Deng is with the Department of Computer Science, University of North Carolina at Greensboro (UNCG), Greensboro, NC 27412. E-mail: jing.deng@uncg.edu.*
- *X. Fan is with the Key Laboratory for Ubiquitous Network and Service Software of Liaoning Province, and also with the DUT-RU International School of Information Science & Engineering, Dalian University of Technology, Dalian 116600, China. E-mail: xin.fan@dlut.edu.cn.*
- *G. Wu is with the Key Laboratory for Ubiquitous Network and Service Software of Liaoning Province, and also with the School of Software, Dalian University of Technology, Dalian 116600, China. E-mail: wgwdut@dlut.edu.cn.*

Manuscript received 16 Nov. 2017; revised 27 Sept. 2018; accepted 6 Oct. 2018. Date of publication 16 Oct. 2018; date of current version 12 Nov. 2020. (Corresponding author: Lin Yao.)
Digital Object Identifier no. 10.1109/TDSC.2018.2876257

In NDN, each router contains three key data structures: Content Store (CS), Pending Interest Table (PIT), and Forwarding Information Base (FIB) [4] (see Fig. 1). CS is used to cache the contents forwarded by the router. PIT records both the interests that have been forwarded but have not yet been replied and the interfaces from which they have arrived. FIB is a table for routing the incoming interest packets based on their name prefixes. When an interest is received, the router first checks its own CS. If the corresponding content is found in its CS, the router will send the data packet towards the interface. Otherwise, if there is an interest match in PIT, the interface will be added into PIT. Instead, if there is a matching entry in FIB, the interest will be forwarded. All other interests are discarded silently. When a data packet is received, the router will decide whether to cache it based on its cache replacement policy.

The above information retrieving procedure makes it attractive for NDN to use in-network caching, which can facilitate the efficient distribution of popular contents, reduce overall latency and improve bandwidth utilization [5]. However, pervasive caching is vulnerable from malicious data request attacks called Cache Pollution Attacks (CPAs) [6], [7]. Different from Denial of Service (DoS) attack [8], [9], [10], [11], [12], CPA attackers do not aim to forge nonexistent content names in order to target a specific content provider or sabotage the network infrastructure. Their goal is to pollute the caching balance by sending a large number of bogus or fabricated interest packets to the network and thereby tricking the NDN routers into caching non-popular contents. There are two types of CPA: location-disruption attack (LDA) and false-locality attack (FLA). In LDA, attackers keep requesting different

1545-5971 © 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on January 23,2025 at 04:08:39 UTC from IEEE Xplore. Restrictions apply.

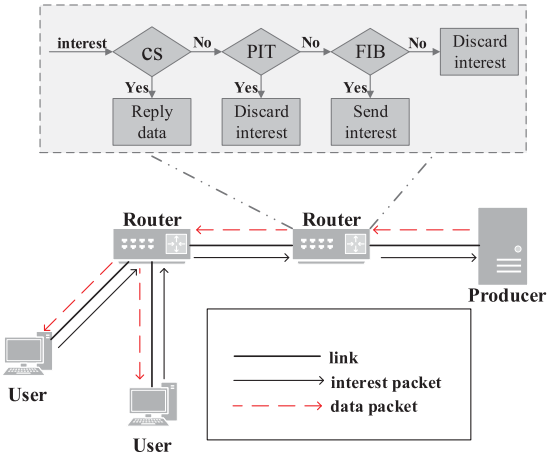


Fig. 1. Data process in NDN.

non-popular contents and squeeze out the popular contents in cache. In FLA, attackers request a set of non-popular contents continually and inject these contents into the cache. Under CPA, the cache hit of content requests from legitimate users is degraded and the content retrieval latency is increased [13].

Despite various efforts, accurate detection and strong defense against CPA remain elusive due to the difficulty of separating different requests. Some detection schemes in the previous works are inaccurate or just simply drop the suspicious packets once they are detected [14], [15], [16]. In light of the above challenges, we propose a Detection and Defense scheme of Cache Pollution based on Clustering (DDCPC) for NDN. Our approach is to use a clustering technique to group past and new interests by computing the statistics and then monitor their arrival statistics. DDCPC detects CPAs by taking advantage of the well-known Zipf-like distribution of legitimate requests [17]. We also consider a particular case that a large number of users suddenly become interested in some non-popular data. In this situation, these requests should be considered as legitimate packets rather than malicious ones. For example, the Nobel prize on Literature was awarded to Mo, Yan in 2012 as the first ever Chinese recipient in the category. The news led to a sharp increase of hits on some of his old works. Without the capability of handling this kind of bursty legitimate requests, they would be treated as attacks. Given all the above considerations, our work focuses on the following contributions:

1) To the best of our knowledge, we are the first to adopt the clustering technique to design and implement a detection scheme. After clustering the requests based on their frequency and the interval time between two consecutive requests for the same content, we can determine CPA based on the clustering result.
2) We design and implement a defense mechanism by generating an attack table to record the abnormal requests whose corresponding data packets are not cached in the NDN routers.
3) To the best of our knowledge, we are the first to consider the special case that a large number of users suddenly become interested in some non-popular contents.
4) DDCPC is compared with three other state-of-the-art techniques CPMH [5], LMDA [16] and ANFIS [13] and the extensive simulations show that our scheme is superior in most of the evaluated performance metrics.

The rest of this paper is organized as follows. In Section 2, we discuss the related work. The preliminary is given in Section 3. We present the details of our DDCPC scheme in Section 4. Simulation results and discussions are presented in Section 5, followed by complexity discussion in Section 6. Finally, we conclude our work in Section 7.

2 Related Work

Though LDA and FLA were first proposed for Internet by Gao et al. in [18], [19], there is very limited work recently on detecting LDA or FLA in NDN since NDN is a new promising architecture of future Internet. In this section, we review some literature works related to our proposed method.

LDA-based. Park et al. [14] proposed a detection approach against LDA for Content-Centric Networking (CCN). Low rate LDA can be detected through randomness check on the distribution of content objects. However, the repeated requests from attackers are ignored in this paper. CacheShield [20] is the first countermeasure for cache pollution attack specifically designed for NDN. When the router receives a content, a shielding function is calculated to decide whether to cache it or not. This shielding function reduces the possibility that non-popular contents are cached. However, some potential popular contents are not cached. In [13], a cache replacement method called ANFIS was presented to mitigate CPA impacts in NDN. ANFIS is an integration of neural network architectures with fuzzy inference system to map a couple of input-output data patterns. The output of each data pattern is a goodness value which determines the type of content ranging. ANFIS can efficiently and accurately mitigate FLA (by removing fake contents) and LDA (by removing those contents with the lower goodness values for cache replacement) in a timely manner. In [15], Xu et al. took advantage of the difference between attack traffic and regular traffic. They assumed that the attackers sent out malicious interests whose names possess the same prefix. In [16], the probability associated with each content object was calculated. By evaluating the probability change, LDA can be detected.

FLA-based. In [15], [16], FLA can also be detected. In [13], the ANFIS-based detection scheme can detect and mitigate FLA. In [21], a ranking algorithm for cached contents was proposed to distinguish good and bad contents in NDN. In [22], the diversity of the interest traversing paths was exploited to detect and mitigate FLA. The paper was designed based on the idea that the malicious requests are unlikely to traverse as many different paths as the legitimate requests. A request is judged as a malicious one and evicted if such a path diversity for this request is not observed. In [5], the malicious contents were determined by computing the weighted request rate variation per prefix and then not stored in each router. In [23], a framework was proposed to detect FLA or LDA by selecting some nodes close to clients as monitoring nodes. These nodes cooperate to capture some information within the network such as the

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on January 23,2025 at 04:08:39 UTC from IEEE Xplore. Restrictions apply.

request rate and hit ratio so as to detect the pollution attack. In [24], a non-collaborative cache allocation scheme was proposed to make caching decision and thereby defend the pollution attack by integrating popularity and locality. The non-collaborative scheme makes contents in each router inconsistent. Therefore, it is difficult for an attacker to master the contents in most routers, which can avoid the pollution attack.

Summary-While there have been some schemes considering LDA or FLA, few can resist both LDA and FLA except [13], [15], [16]. In [13], ANFIS adopted the idea of artificial neural network to prevent LDA and FLA. The heavy computation makes it difficult to be implemented in the realistic NDN. ELDA was proposed based on the assumption that the malicious requests possess the same name prefix [15]. In [16], no defense scheme was designed. Compared with the existing works, our approach can capture the abnormal behaviors and detect both LDA and FLA by adopting the clustering technique. To better serve the legitimate requests, our defense algorithm considers a specific but often observed case where a large number of users suddenly become interested in some non-popular data.

3 Preliminary

Clustering, one of the most important methods of unsupervised learning, deals with the data structure partition in an unknown area [25]. A cluster is therefore a collection of elements with higher similarity. In our DDCPC scheme, we adopt the clustering algorithm in [26] to classify all the requests based on the local density of data points. In this algorithm, cluster centers are surrounded by neighbors with lower local density and they are at relatively long distances from all points with a higher local density. The clustering algorithm includes two phases, computation of the cluster center and adjustment of clusters.

Phase 1. Before computing the cluster center, we first calculate the local density ρ_i of the ith point p_i,

$$\rho_i = \sum_j \chi(d_{ij} - d_c), \tag{1}$$

where d_{ij} is the distance between p_i and p_j, and d_c is the cutoff distance to determine the local density as a threshold (we set d_c to make the average number of neighbors 2 percent of the total number of points in the data set, as recommended in [26].) The function $\chi(x)$ is defined as $\chi(x) = \begin{cases} 1 & \mathrm{x} < 0 \\ 0 & \mathrm{x} >= 0. \end{cases}$

Eq. (1) also shows ρ_i is the number of points whose distances from p_i are less than d_c. If p_i possesses the highest density ρ_i, δ_i is defined as the largest distance from p_i and any other point with a lower local density,

$$\delta_i = \max_{j:\rho_j < \rho_i} (d_{ij}). \tag{2}$$

Else, δ_i is defined as the smallest distance from p_i to any other point with a higher local density,

$$\delta_i = \min_{j:\rho_j > \rho_i} (d_{ij}). \tag{3}$$

Generally, it is necessary that each cluster center has a large $\theta = \rho \cdot \delta$.

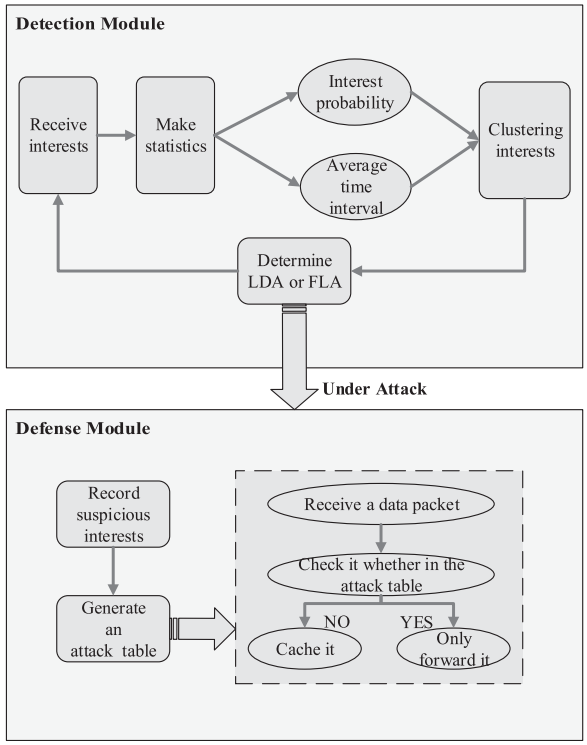


Fig. 2. Architecture of DDCPC.

Phase 2. After the cluster center is computed, each point will be classified into the closest cluster. c_i is defined as the cluster that p_i belongs to,

$$c_i = \begin{cases} k & p_i \text{ is the } k\text{-th cluster center} \\ c_j & x_j \text{ is the nearest neighbor of } x_i \text{ with a larger } \rho \text{ than } x_i \end{cases}. \tag{4}$$

Similarly, each remaining point is classified into the same cluster with its nearest neighbor which has a higher density.

4 Detection Mechanism Based On Clustering

We would like to exploit the clustering technique to detect CPA based on the distribution change of interests. We first give an overview of the scheme, and then present our DDCPC scheme in details.

4.1 Overview

Our DDCPC scheme aims to detect CPA and then start the defensive measure to mitigate the damage. It mainly contains two modules: detection of CPA and defense of CPA (see Fig. 2). With CPA, both LDA and FLA will distort the Zipf-like distribution which legitimate requests usually follow. When LDA is launched, the attackers normally request all contents at an equal probability, which makes the distribution flat. Under FLA, the adversaries select a subset of non-popular contents and issue interests for this subset. Correspondingly, the percentage of non-popular requests increases sharply, which changes the distribution of interests too. An overview of how DDCPC works is as follows:

1) As soon as an interest is received, the router will update the popularity table for this interest and make

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on January 23,2025 at 04:08:39 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Frequent Notations

Notation	Description
p_i	The ith point
c_i	The content object c_i
$p(c_i)$	The interest probability for c_i
N	The total number of interests in a slice
ϵ	The detection slice
ϱ	The biggest interval
$t_m(c_i)$	The time interval between the mth and $(m+1)$th interests for c_i
$t(c_i)$	The average time interval between two consecutive requests for c_i
ω	The transfer rate
ξ	The threshold value

the following statistics, the number of interests, number of interests for the same content, and time interval between two consecutive requests for the same content.

2) Periodically, all the received interests are classified into different clusters based on the interest probability and the average time interval between two interests for the same content.
3) Based on the clustering result, we can determine whether the router is under LDA or FLA.
4) Once LDA or FLA is detected, an attack table will be generated to record those suspicious interests and broadcasted to the neighbors hop by hop. All the routers receiving the attack table will not cache the content chunks in the corresponding data packets.

4.2 Detection of CPA

This module aims to detect LDA or FLA. First, we list frequently used notations in Table 1 and some terminologies are defined. Then, the detection algorithm will be presented.

Definition 1 (Interest Probability). *It is defined as the ratio between the number of interests with the same name prefix and the total number of all the interests in a slice. $p(c_i)$ represents the interest probability for c_i,*

$$p(c_i) = n(c_i)/N, \tag{5}$$

where c_i is the ith content object, $n(c_i)$ is the total number of interests for c_i in a slice, and N is the total number of interests in that slice.

Definition 2 (Time interval). *The average time interval between two consecutive requests for c_i is defined as*

$$t(c_i) = \begin{cases} \frac{\sum_{m=1}^{n(c_i)-1} t_m(c_i)}{n(c_i)-1} & n(c_i) >= 2 \\ rand(\varrho, \epsilon) & n(c_i) = 1, \end{cases} \tag{6}$$

where $t_m(c_i)$ is the mth time interval between the mth and $(m+1)$th interests for c_i in the slice T_p. When there is only one interest for c_i one time slice, it will be considered as non-popular and the accurate time interval cannot be computed directly. To facilitate clustering, we assign an artificially large value of time interval in order to classify it into the non-popular cluster. In order to distinguish from the interest for c_i of the next slice, a random value between ϱ and ϵ is assigned, where ϱ is the biggest interval and ϵ is the slice value.

Algorithm 1. Interest Metrics Computing

Input:
ϵ ▷ duration of each time slice
N ▷ number of different contents
Output:
m ▷ array to store request ratios
ψ ▷ array to store average time interval for interests
1: **for** receive an interest for content i at t **do**
2: $M[i] \leftarrow M[i] + 1$
3: $count \leftarrow count + 1$
4: $\Psi[i] \leftarrow \Psi[i] + t - t_l$
5: $t_l = t$
6: **end for**
7: **for** $i = 1 \rightarrow N$ **do**
8: $m[i] \leftarrow M[i]/count$
9: **if** $M[i] = 1$ **then**
10: $\psi[i] \leftarrow rand(\varrho, \epsilon)$
11: **else**
12: $\psi[i] \leftarrow \Psi[i]/(M[i] - 1)$
13: **end if**
14: **end for**
15: **return** m and ψ

In DDCPC, we use the following steps to detect LDA and FLA:

Step(1). In Algorithm 1, we first compute the total number of all interests, total count of the interests for the same content c_i and time interval of two consecutive interests for the same content c_i in one slice on lines 1 to 6, where M is an array to store the number of interests for c_i, and t_l is the time when last interest for c_i is received. Then, each NDN router computes *Interest Probability* and *Time interval* for c_i on lines 7 to 14.

Step(2). In Algorithm 2, we compute euclidean distance ε between all data points on lines 1 to 7 and κ is a weight coefficient. The local density ρ and distance δ of each data point are computed on lines 8 to 21. On lines 22 to 29, all points are clustered. ϑ represents the number of cluster centers and ϑ_{mark} marks the cluster for each point.

Step(3). Algorithm 3 is used to detect whether FLA or LDA has been launched based on the clustering result. As discussed before, the two clusters are merged into one under LDA on lines 1 to 2 and the number of requests in the non-popular cluster suddenly drops and that in the popular cluster increases under FLA on lines 4 to 11. To identify the percentage of points from the non-popular cluster to the popular one, we define the transfer rate ω

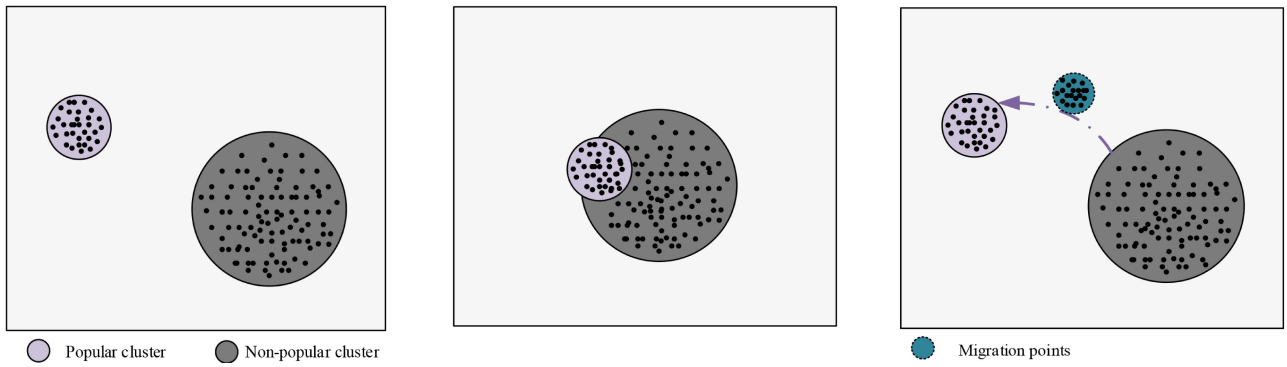
$$\omega = \frac{n_t}{\tau}, \tag{7}$$

where τ is the number of points in the popular cluster and n_t is the number of moving points from the non-popular cluster to the popular cluster. Under FLA, ω will increase dramatically. We define the threshold of ω_i as follows:

$$\xi = \lambda \frac{\sum_{i=1}^{N} \omega_i}{N}, \tag{8}$$

where N is the number of slices, ω_i represents the transfer rate in the ith slice, $\frac{\sum_{i=1}^{N} \omega_i}{N}$ represents the average value

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on January 23,2025 at 04:08:39 UTC from IEEE Xplore. Restrictions apply.



(a) normal clusters (b) clusters under LDA (c) clusters under FLA

Fig. 3. Clusters of different cases.

of all the measured ω_i and λ is an adaptation parameter to control the threshold. As ω_i changes slightly without any attack, the threshold is set as λ times of the average value to detect the attack and reduce the false alarm.

Algorithm 2. Clustering

Input:
m ▷ array to store request ratios
ψ ▷ array to store average inter-arrival times for interests
N ▷ number of contents

Output:
ϑ ▷ number of clusters
ϑ_{mark} ▷ array of cluster mark of each data point

for $i = 1 \rightarrow N$ **do**
  for $j = i + 1 \rightarrow N$ **do**
    $\zeta = m[i] - m[j]$
    $\eta = \psi[i] - \psi[j]$
    $\varepsilon[i][j] = \sqrt{\zeta^2 + \kappa * \eta^2}$
  end for
end for
for $i = 1 \rightarrow N$ **do**
  for $j = 1 \rightarrow N$ **do**
    if $\varepsilon[i][j] < d_c \&\& i! = j$ **then**
      $\rho[i] \leftarrow \rho[i] + 1$
    end if
  end for
end for
for $i = 1 \rightarrow N$ **do**
  if $\rho[i]$ equals the max value of ρ **then**
    $\delta[i]$ equals the max distance between i and other points
  else
    $\delta[i]$ equals the minimum distance between i and other points with larger ρ
  end if
end for
for $i = 1 \rightarrow N$ **do**
  if i *is cluster center* **then**
    $\vartheta \leftarrow \vartheta + 1$
    $\vartheta_{mark}[i] = \vartheta$
  else
    $\vartheta_{mark}[i]$ equals the cluster mark of its nearest neighbor (with larger ρ)
  end if
end for
31: **return** ϑ and ϑ_{mark}

Algorithm 3. CPA Detection

Input:
ϑ ▷ number of clusters
ϑ_{mark} ▷ array of cluster mark of each data point
N ▷ number of all data points
τ ▷ number of points in popular cluster in last time slice
ξ ▷ threshold of transfer rate

Output:
$decision$

if $\vartheta = 1$ **then**
  $decision = LDA$
else
  for $i = 1 \rightarrow N$ **do**
    if $\vartheta_{mark}[i] = 1$ **then**
      $\phi \leftarrow \phi + 1$
    else
      $\varphi \leftarrow \varphi + 1$
    end if
  end for
  $\omega \leftarrow |\phi - \tau|/\tau$
  if $\omega \leq \xi$ **then**
    $decision \leftarrow NoAttack$
  else
    $decision \leftarrow FLA$
  end if
end if
return $decision$

4.3 Defense of CPA

Once LDA or FLA is detected, the defense mechanism will be activated. The purpose of CPA is to make the router cache some non-popular contents instead of popular ones. Our defense mechanism is to prevent the malicious contents from being cached. To achieve it, we generate an attack table to record the abnormal interest and the corresponding contents will not be cached in CS. In DDCPC, we use the following steps to defend CPA:

Step(1). Normally, clustering generates popular and non-popular clusters as it is illustrated in Fig. 3a. The points in the non-popular cluster are recorded. Under LDA, the distribution of interests is flatter, which shows some popular and non-popular requests have mixed together as it is illustrated in Fig. 3b. The content names corresponding the points in the non-popular clustering are added into the attack table.

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on January 23,2025 at 04:08:39 UTC from IEEE Xplore. Restrictions apply.

TABLE 2
Simulation Parameters

Simulation Parameters	Value
Link (Router to Router)	50 Mb/s
Link Delay (Router to Router)	5 ms
Number of contents	10,000
CS Size	100
PIT Size	12,000
Content Size	1 MB
Cache Strategy	LRU
Legitimate Request	3,000 interests/s
M	total number of content items
λ	5.2

Step(2). Under FLA, only a subset of non-popular contents are requested continuously, which causes some points in the non-popular cluster to move into the popular cluster as it is illustrated in Fig. 3c. The content names corresponding to these points are added into the attack table.

During designing the defensive measure, we cannot simply adopt the policy of discarding the interests because we consider the particular case that a large number of users may suddenly become interested in some non-popular data. In this case, the content chunks should be returned to the requesting nodes normally. Consequently, our solution is forwarding all the interests but not caching the content trunks corresponding to abnormal interests. In this situation, regardless of whether the interests are suspicious, each interest will be forwarded until the time-to-live in the packet expires or it reaches the content provider. Data packets can be replied normally to satisfy the special users.

5 Performance Evaluation

In order to evaluate the performance of our DDCPC scheme, we conduct our simulations in ndnSIM [27], a popular open-source NDN simulator based on NS-3. All the experiments are simulated in ndnSIM1.0 on a local computer equipped with an Intel Core i7 2.50 GHz CPU, 8 GB RAM and Ubuntu 16.04 LTS. All the primary parameters used in ndnSIM are provided in Table 2. In our simulations, the total number of the content items M are 10,000 and the cache size is equal to the 1 percent of M. Each content item has the same size (1 MB). The PIT size of each router is 12,000 entries. The cache replacement policy is Least Recently Used (LRU) [28], which is commonly used by in-network caching.

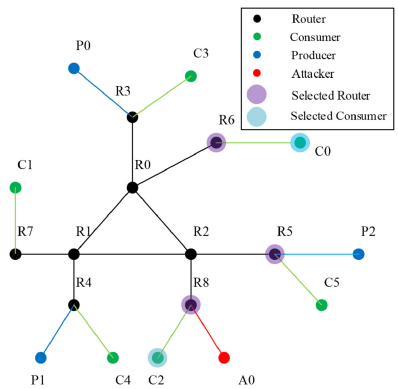


Fig. 4. XC topology.

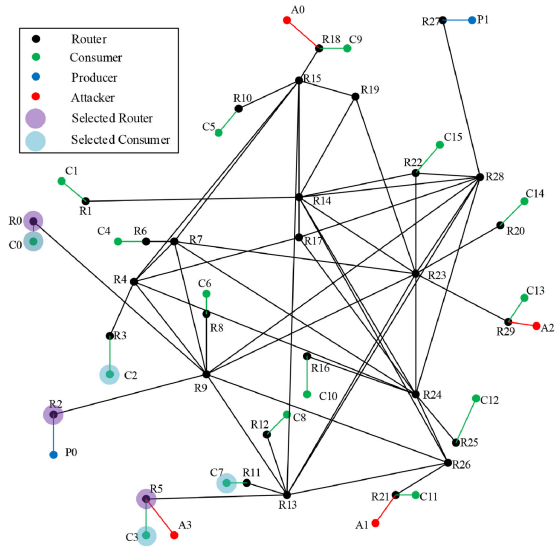


Fig. 5. DFN topology.

For our simulations, we consider the following three topologies, XC topology, DFN topology [5], [16], and AS-3967 network topology [23], as illustrated in Figs. 4, 5, 6 respectively. XC topology is a simple one but includes all of the necessary components of a useful network, helpful for us to understand the inner-working of different schemes. DFN topology is a more complex and realistic topology than XC. AS-3967 network topology is an actual ISP network topology snapshot. In XC, there are 6 legitimate consumers and one attacker. In DFN, there are 16 legitimate consumers and 4 attackers. Besides 129 nodes and 147 edges in AS-3967 topology, we add 44 legitimate consumers and 6 attackers in our simulations.

All our simulations span over 24 hours, and follow a similar pattern. During the first 12 hours, all interests are issued by legitimate users. We set the rate of legitimate consumers requesting contents as 3,000 interests per second, which follows the Zipf-like distribution. Then, adversaries launch attacks starting at the 12th hour.

5.1 Performance Metrics

Our main comparisons are made between the proposed DDCPC scheme, CPMH [5], LMDA [16] and ANFIS [13]. Our DDCPC scheme detects CPA by clustering interests.

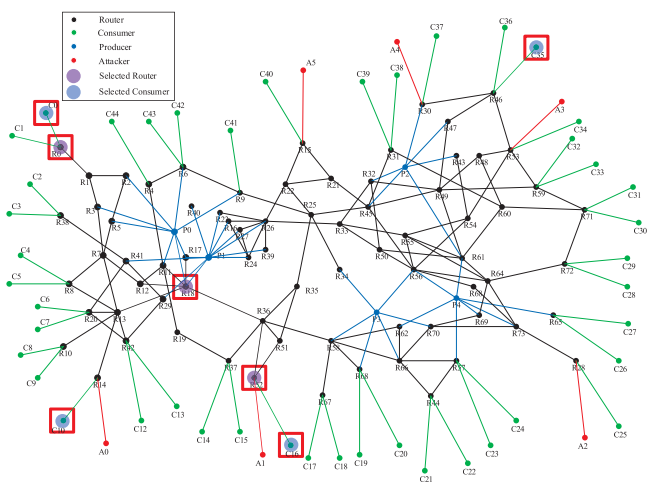


Fig. 6. AS-3967 topology.

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on January 23,2025 at 04:08:39 UTC from IEEE Xplore. Restrictions apply.

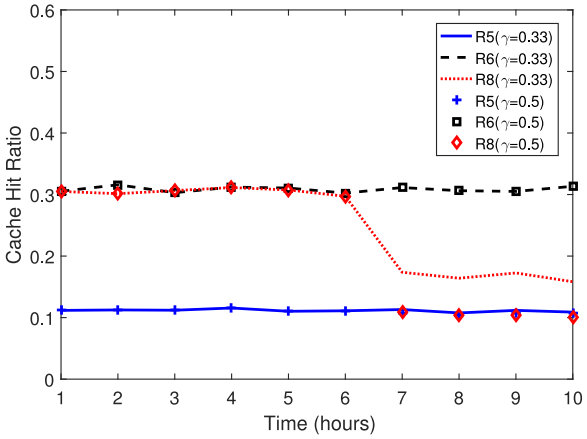


Fig. 7. Cache hit ratio under LDA in XC.

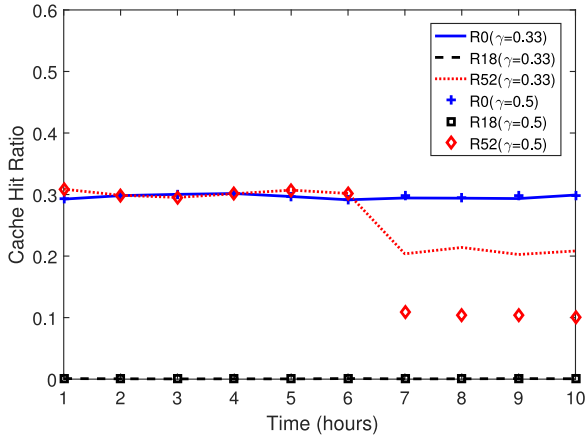


Fig. 8. Cache hit ratio under LDA in AS-3967.

Based on the distribution of interests, CPA is detected and the corresponding defense is implemented. In LMDA, the request probability associated with each content object is calculated. By evaluating the probability variation, LDA or FLA is detected. CPMH scheme makes use of hierarchy of content name prefixes to detect CPA. If there is a large aggregation of some contents, the attack might have been launched. ANFIS uses neural network with a fuzzy inference system to remove fake contents and those with low goodness values. In our simulations, we mainly use the following performance metrics:

- *Detection Ratio*, the ratio of detecting attacks. It is defined as the ratio between the number of correctly detected attacks and the total number of attacks.
- *Cache Hit Ratio*, the ratio of successful queries. It is defined as the ratio between the number of queries successful replied and all of the queries.
- *Average Hop Count*, the average number of hops through which a legitimate user receives the corresponding data.
- *False Alarm Ratio*, the ratio of false detection. It is defined as the ratio between the number of falsely detecting regular content interests as attacks and the total number of detections.

5.2 Impacts of CPA

In this section, we evaluate the damage effect from LDA and FLA. We choose three kinds of routers, the upstream router, the router connecting to the normal users and the router connecting to the attackers. In Fig. 4, we choose three routers $R5$, $R6$ and $R8$ to evaluate the hit ratio under attacks. In Fig. 5, we choose $R0$, $R2$ and $R5$ to evaluate the hit ratio. In Fig. 6, we choose $R0$, $R18$ and $R52$ to evaluate the hit ratio. As Table 2 shows the LinkDelay (Router to Router) is 5ms, we use the average hop count to evaluate the delay from sending an interest to receiving the corresponding data. We choose two consumers $C0$ and $C2$ in XC topology, four consumers $C0$, $C2$, $C3$ and $C7$ in DFN topology and four consumers $C0$, $C10$, $C16$ and $C35$ in AS topology. Three attackers, $A0$ as it is illustrated in Fig. 4, $A3$ as it is illustrated in Fig. 5 and $A1$ as it is illustrated in Fig. 6 are connecting to the same routers $R8$, $R5$ and $R52$ with $C2$, $C3$ and $C16$, respectively. We choose 10 hours of the simulations to show the results and the attacks are launched after the 6th hour.

LDA Attack. In LDA, we design the following scenario that the attackers only request all existing contents without generating any new content. γ is defined as the attack intensity, the ratio between malicious interests and the total number of interests.

Fig. 7 shows the cache hit ratio under LDA in XC topology. Before the attack is launched, three routers have a constant hit ratio. Once the attack is launched after the 6th hour, the hit ratio of $R8$ decreases sharply because of its direct connection to the attacker $A0$. The hit ratio of $R5$ still remains steady after LDA happens. The attack has no effect on the upstream router $R6$ because it has a distance of three hops from $A0$. From Fig. 7, we can conclude that $R8$ has the smallest hit ratio as γ increases, signifying a higher impact. DFN network has the similar results on the hit ratio but we omit it due to the page limit.

Fig. 8 shows the cache hit ratio under LDA in AS-3967 topology. Similarly, three routers have a constant hit ratio without any attack. Once LDA is launched, the hit ratio of $R52$ decreases sharply because it directly connects to the attacker $A1$. And $R52$ gets a smaller hit ratio when γ increases. The hit ratio of $R0$ and $R18$ remains steady even with LDA attacks, showing that LDA only has some impacts on routers closely connected to the attackers.

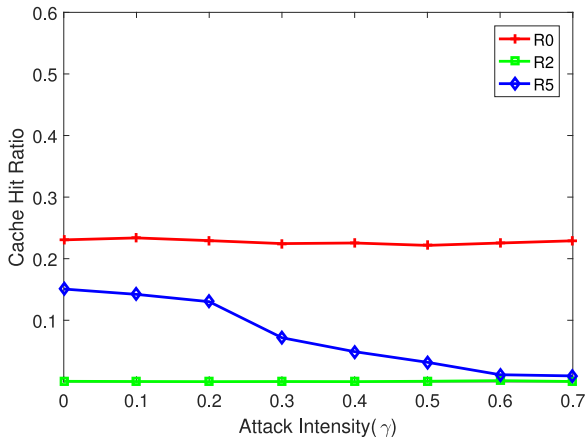


Fig. 9. Cache hit ratio under LDA in DFN.

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on January 23,2025 at 04:08:39 UTC from IEEE Xplore. Restrictions apply.

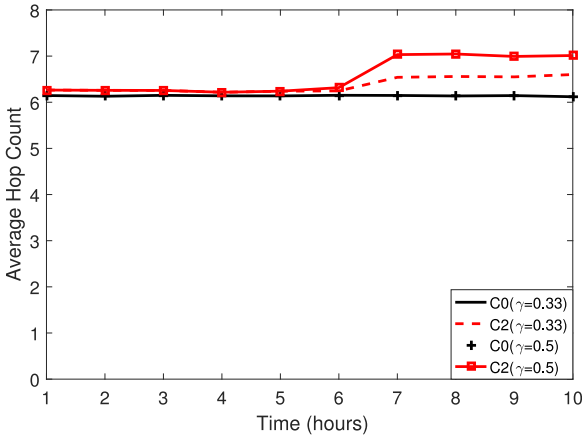


Fig. 10. Average hop count under LDA in XC.

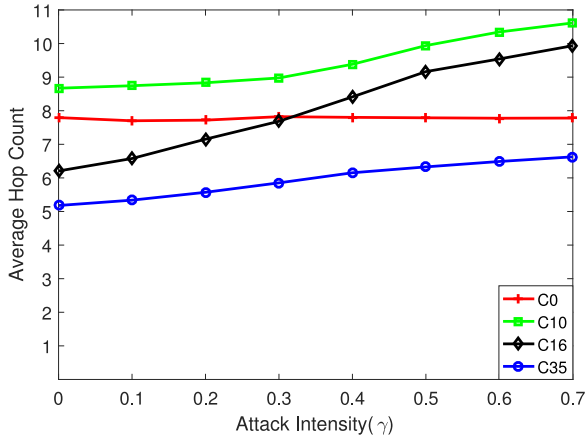


Fig. 12. Average hop count under LDA in AS-3967.

Fig. 9 shows the average cache hit ratio under LDA in DFN topology as γ increases. $R0$ has a stable caching ratio about 0.22 compared with $R5$ and $R2$ because no attacker is connecting to it and it has a higher number of hops from the attacker. The hit ratio of $R2$ is almost zero because it only connects to a producer and receives few requests. The hit ratio of $R5$ degrades gradually as γ increases because non-popular contents are cached instead of popular ones under LDA.

Fig. 10 shows the average hop count of selected consumers under LDA. For the normal consumer whose router does not connect with any attacker, $C0$ as it is illustrated in Fig. 10, the hop count remains relatively stable. For the normal consumer $C2$, the hop count increases once the attack is launched (though only by 10 percent). And the hop count of $C2$ is higher as γ increases.

Fig. 11 shows the average hop count of $C0$, $C2$, $C3$, and $C7$ under LDA in DFN topology. The hop count of $C0$ remains at about 6.6 because it is not next to any attacker. The hop count of $C3$ is 8.2 with $\gamma = 0$ (no attack). When LDA happens, the hop count of $C3$ starts to increase and it increases more as γ increases. The hop count can be up to about 10 with $\gamma = 0.7$, a 20 percent increase.

Fig. 12 shows the average hop count of $C0$, $C10$, $C16$, and $C35$ under LDA in AS-3967 topology. Similar to the results in DFN topology, the hop count of consumer $C0$, which is far away from the attackers, remains steady. The hop count of $C16$ is 6.2 without LDA, but increases to 9.9 with attack intensity $\gamma = 0.7$. Similar patterns can be observed in the hop counts of $C10$ and $C35$, though with slightly slower rate of increase.

FLA Attack. In FLA, we assume that the attackers only request existing contents and are aware of a few contents that are not popular at all. Thus, they only request a subset of least-request contents that are on the long tail of the Zipf-like distribution. Such a ratio is set to 0.1 in our evaluations.

Fig. 13 shows the cache hit ratio from FLA in XC topology. The hit ratio of $R6$ remains at 0.3 because it has a distance of three hops from the attacker $A0$ and has not been affected. Similarly, the hit ratio of $R5$ remains steady at about 0.11. $R8$ degrades to about 1/3 once FLA is launched because the attacker $A0$ is directly connected to $R8$.

Fig. 14 shows the cache hit ratio from FLA in AS-3967 topology. The hit ratios of $R0$ and $R18$ remain steady because they are not connected to any attacker directly. Hit ratio of $R52$, connecting to $A1$, degrades from 0.3 to 0.2 with $\gamma = 0.33$ and to 0.1 with $\gamma = 0.5$ for FLA.

Fig. 15 shows the cache hit ratio under different γ in DFN topology. $R0$ has a stable caching ratio around 0.22 whether FLA happens or not, because it has a large number of hops from attackers. The hit ratio of $R2$ approaches to zero because it receives few requests. $R5$ is directly connecting to the attacker $A0$, causing its hit ratio to drop as γ increases.

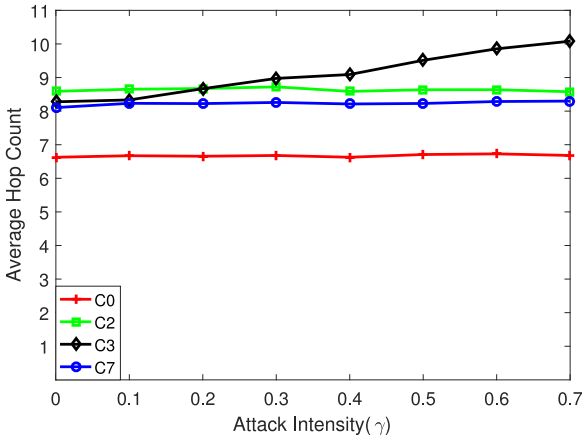


Fig. 11. Average hop count under LDA in DFN.

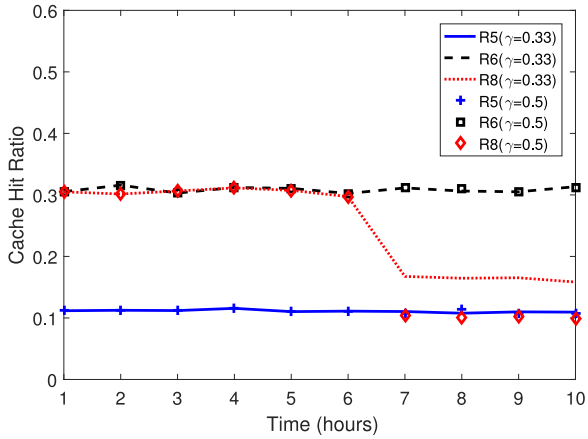


Fig. 13. Cache hit ratio under FLA in XC.

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on January 23,2025 at 04:08:39 UTC from IEEE Xplore. Restrictions apply.

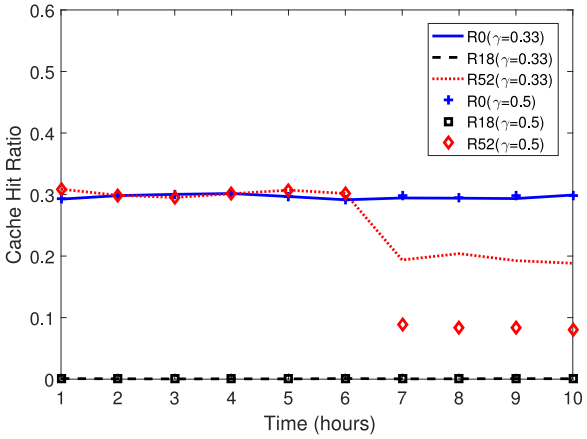


Fig. 14. Cache hit ratio under FLA in AS-3967.

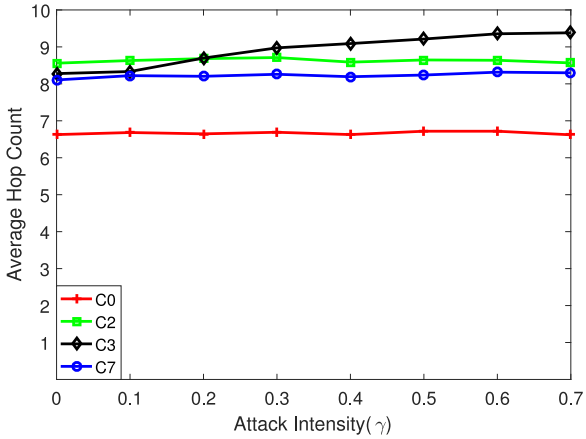


Fig. 16. Average hop count under FLA in DFN.

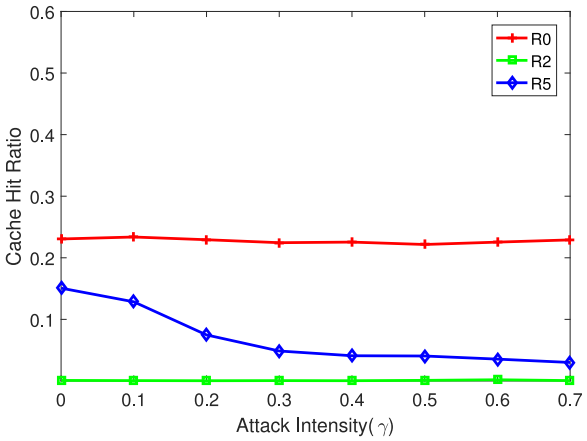


Fig. 15. Cache hit ratio under FLA in DFN.

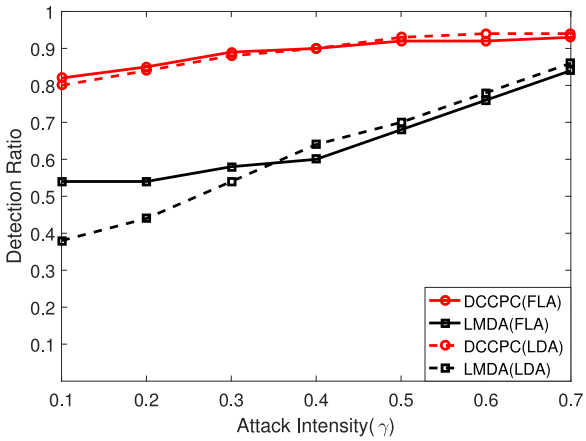


Fig. 17. Detection ratio under CPA.

Fig. 16 shows the average hop count under different γ in DFN topology. The hop count of $C0$ remains stable at 6.6. The hop count of $C3$ is 8.2 when there is no attack. It increases with γ and reaches 9.4 with $\gamma = 0.7$.

5.3 Performance of DDCPC

In this section, we compare our DDCPC scheme with CPMH and LMDA in terms of detection ratio, cache hit ratio, and average hop count. Then, we evaluate the false alarm ratio and detection ratio by tuning the parameter λ which controls the threshold of transfer rate.

Detection Ratio. Because CPMH adopts the detection of LMDA [5], we only evaluate the detection ratio of DDCPC and LMDA. We adopt the same mechanism to detect any attack at the edge router such as $R8$ in XC and $R5$ in DFN. We show the detection ratio of $R5$ below ($\gamma = 0.33$).

Fig. 17 shows that our DDCPC scheme maintains a much higher detection ratio under the different γ. The ratio of LMDA increases with γ. The detection ratio is lower than 0.6 when γ is smaller than 0.4. In LMDA, the threshold computed by the change of requesting contents is used to detect the attacks. The detection ratio rises naturally with γ. Fig. 17 shows that DDCPC is superior to LMDA in detection ratio.

Cache Hit Ratio. Fig. 18 presents cache hit ratios for $R52$ in AS-3967 and $R5$ in DFN under attacks. DDCPC outperforms ANFIS by about 8 percent when in networks with FLA, while results in similar hit ratios in networks with LDA.

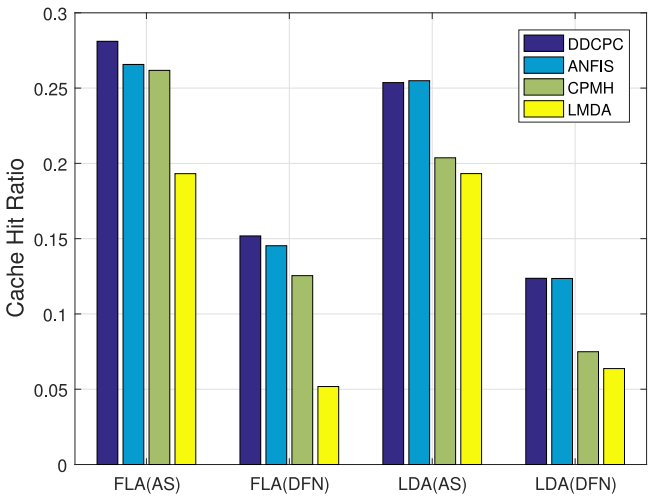


Fig. 18. Hit ratio under CPA.

DDCPC and ANFIS perform better than CPMH and LMDA in networks with either attacks. LMDA experiences the worst hit ratios, due to its lack of defense against such attacks. While, DDCPC outperforms ANFIS for 8 percent under FLA but offers the same performance under LDA. Under FLA, the attackers choose a subset of non-popular contents and request this set continuously. In this case, more abnormal requests can be accurately assigned to the

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on January 23,2025 at 04:08:39 UTC from IEEE Xplore. Restrictions apply.

TABLE 3
Average Hop Count under CPA

Algorithm	After attack (DFN)	After attack (AS-3967)
$LMDA(FLA)$	9.1	7.2
$CPMH(FLA)$	8.5	6.4
$ANFIS(FLA)$	8.4	6.4
$DDCPC(FLA)$	8.3	6.2
$LMDA(LDA)$	9.1	7.2
$CPMH(LDA)$	9.1	7.0
$ANFIS(LDA)$	8.4	6.4
$DDCPC(LDA)$	8.4	6.4

attack table compared with the case that distinct non-popular contents are continuously requested under LDA. ANFIS mitigate FLA and LDA by removing those contents with the lower goodness values for cache replacement.

Average Hop Count. We select $C3$ in DFN and $C16$ in AS-3967 to evaluate the average hop count. We choose these consumers which are strongly influenced by CPA, because they are next to the attackers.

Table 3 shows the hop count of $C3$ in DFN and that of $C16$ in AS-3967 under CPA. Our DDCPC scheme is consistently better than LMDA and CPMH, sometimes even slightly better than ANFIS.

Fig. 19 shows the false alarm ratio and detection ratio of our DDCPC scheme in DFN under CPA with different λ. When λ is smaller than 4, the detection ratio is 1, but the false alarm ratio is always bigger than 0.2. And when we choose a bigger λ, the false alarm ratio and detection ratio both get smaller. When λ is between 4.8 and 5.6, the false alarm ratio becomes almost to zero and the detection ratio is bigger than 0.9. So we should choose an appropriate parameter to get a better performance. In our simulation, we choose λ to be 5.2.

6 Algorithm Complexity Analysis

We compare the algorithm complexity of our DDCPC scheme and the other two state-of-the-art schemes and summarize the results in Table 4.

LMDA detects CPA by calculating the probability of content object. It computes the variation of the probability to determine whether CPA happens. CPA may happen if the variation exceeds the maximum acceptable variability. The processes of computing the probability and threshold

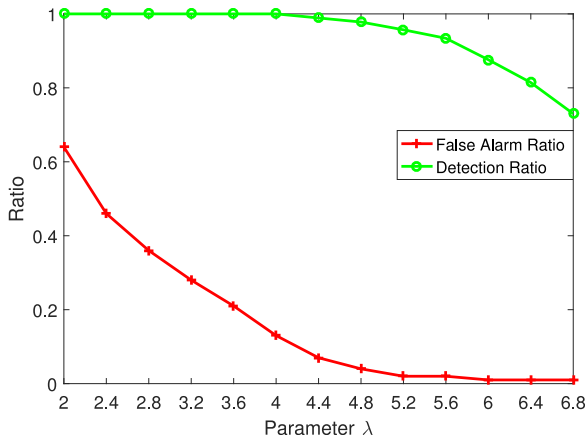


Fig. 19. False alarm and detection ratio with λ in DFN.

TABLE 4
Algorithm Complexity

Algorithm	Complexity
$LMDA$	$O(n)$
$CPMH$	$O(n)$
$DDCPC$	$O(n^2)$
$ANFIS$	$O(n^3)$

both incur complexities of $O(n)$. Thus, the overall complexity of LMDA is $O(n)$.

CPMH uses the same idea of LMDA in the detection phase. The complexity of this process is $O(n)$ like LMDA. CPMH protects the cache by calculating the weighted request rate variation per prefix to identify the attackers' prefixes. And this process incurs a complexity of $O(n)$. So the overall complexity of CPMH is $O(n)$.

Our DDCPC scheme detects CPA by clustering the requests based on their frequency and the interval time between two consecutive requests for the same content. The complexity of computing frequency and interval time is $O(n)$. In the process of clustering, we compute the distance between different points and the complexity is $O(n^2)$. So the overall of our DDCPC scheme is $O(n^2)$.

ANFIS is an integration of neural network architectures with fuzzy inference system to map a couple of input to output data patterns. It uses both the gradient descent and the least squares method to train and adjust the premise and consequent parameters. Thus, the complexity of training process is $O(n^3)$. Trained ANFIS model could output a goodness value between 0 and 1 corresponding to different data pattern. Router updates goodness value for every content periodically and removes contents whose goodness value is lower than threshold. Remaining contents are sorted in ascending order of goodness value. This process incurs a complexity of $O(n)$. When a new content is cached, the router simply removes the content with lowest goodness value and stores it. This process incurs a complexity of $O(1)$. So the overall complexity of ANFIS is $O(n^3)$.

We summarize the complexity results in Table 4. Note that n is usually a relatively smaller number, so the $O(n^3)$ complexity of ANFIS scheme is not prohibitive. Compared with it, DDCPC has a lower complexity.

Complexity Evaluation. To verify the complexity, we implement some experiments on processing time, CPU usage rate and memory usage. We only compare DDCPC with CPMH and ANFIS, because LMDA cannot provide a defense strategy and CPMH uses the same idea of LMDA in the detection phase.

Processing time is measured by the time consumption of handling 10,000 packets. As shown in Table 5, our DDCPC scheme needs $0.134s$, CPMH needs $0.046s$ and ANFIS takes the most time $0.736s$. We also measure CPU usage rate and

TABLE 5
Complexity Evaluation

Algorithm	Processing Time	CPU Usage	Memory Usage
$DDCPC$	0.134 s	1.253%	166.65 MB
$CPMH$	0.046 s	0.898%	23.64 MB
$ANFIS$	0.736 s	1.884%	223.32 MB

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on January 23,2025 at 04:08:39 UTC from IEEE Xplore. Restrictions apply.

memory usage in Table 5. We set the requesting rate to 3,000 packets per second. CPMH uses 0.9 percent of CPU, our DDCPC scheme uses 1.25 percent of CPU and ANFIS uses 1.88 percent of CPU. As running time is longer, ANFIS's memory occupancy is up to more than 223 MB. While, our DDCPC scheme occupies 167 MB memory and CPMH occupies 24 MB memory.

As can be seen in Table 5, the CPMH scheme has the lowest complexity and ANFIS is the most complex among the three. Our DDCPC scheme has a performance better than ANFIS in most cases, as shown in Section 5, with a significantly lower complexity.

7 Conclusion

In this work, we have proposed DDCPC, an efficient detection and defense scheme using the clustering technique to mitigate the damage effect of CPA. We have analyzed the difference between the distribution of regular and malicious requests. Our design is based on clustering the requests to capture the abnormal distribution once FLA or LDA is launched. In our DDCPC scheme, a defense technique is also implemented to maintain an attack table to record the corresponding contents of abnormal requests and only cache the contents that are not indexed in the table.

Our extensive simulations have demonstrated the superior performance of the DDCPC scheme in detection ratio (up to 20 percent gain), cache hit ratio (up to 60 percent gain), and average hop count (up to 10 percent gain). Compared to ANFIS on algorithm complexity, our DDCPC scheme is also superior in processing time (about 80 percent shorter), CPU usage (about 1/3 better), and memory usage (about 1/4 better)

In our future work, we plan to focus on detection and defense of attacks with temporal differences.

Acknowledgments

This research is sponsored in part by National Key Research and Development Program of China (2017YFC0704200) and the National Natural Science Foundation of China (contract/grant numbers: 61772113 and 61872053). Deng's research is supported in part by NSF grant CCF-1320428.

References

[1] B. Ahlgren, C. Dannewitz, C. Imbrenda, and D. Kutscher, "A survey of information-centric networking," *IEEE Commun. Mag.*, vol. 50, no. 7, pp. 26–36, Jul. 2012.

[2] L. Zhang, A. Afanasyev, J. Burke, V. Jacobson, K. Claffy, P. Crowley, C. Papadopoulos, L. Wang, and B. Zhang, "Named data networking," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 3, pp. 66–73, 2014.

[3] A. Seetharam, A. Seetharam, and A. Seetharam, "On caching and routing in information-centric networks," *IEEE Commun. Mag.*, vol. 56, no. 3, pp. 204–209, Mar. 2018.

[4] M. Amadeo, C. Campolo, A. Molinaro, and G. Ruggeri, "Content-centric wireless networking: A survey," *Comput. Netw.*, vol. 72, no. 7, pp. 1–13, 2014.

[5] T. Kamimoto, K. Mori, S. Umeda, and Y. Ohata, "Cache protection method based on prefix hierarchy for content-oriented network," in *Proc. IEEE Consum. Commun. Netw. Conf.*, 2016, pp. 417–422.

[6] E. G. Abdallah, H. S. Hassanein, and M. Zulkernine, "A survey of security attacks in information-centric networking," *IEEE Commun. Surv. Tut.*, vol. 17, no. 3, pp. 1441–1454, Jul.–Sep. 2015.

[7] Q. Li, P. P. C. Lee, P. Zhang, P. Su, H. Liang, and K. Ren, "Capability-based security enforcement in named data networking," *IEEE/ACM Trans. Netw.*, vol. 25, no. 5, pp. 2719–2730, Oct. 2017.

[8] H. Dai, Y. Wang, J. Fan, and B. Liu, "Mitigate DDoS attacks in NDN by interest traceback," in *Proc. Comput. Commun. Workshops*, 2013, pp. 381–386.

[9] A. Afanasyev, P. Mahadevan, I. Moiseenko, E. Uzun, and L. Zhang, "Interest flooding attack and countermeasures in named data networking," in *Proc. IFIP Netw. Conf.*, 2013, pp. 1–9.

[10] A. Compagno, M. Conti, P. Gasti, and G. Tsudik, "Poseidon: Mitigating interest flooding DDoS attacks in named data networking," in *Proc. 38th Annu. IEEE Conf. Local Comput. Netw.*, 2013, pp. 630–638.

[11] N. Tan, R. Cogranne, and G. Doyen, "An optimal statistical test for robust detection against interest flooding attacks in CCN," in *Proc. IFIP/IEEE Int. Symp. Integr. Netw. Manage.*, 2015, pp. 252–260.

[12] K. Ding, Y. Liu, H. Cho, H. Chao, and T. K. Shih, "Cooperative detection and protection for interest flooding attacks in named data networking," *Int. J. Commun. Syst.*, vol. 29, no. 13, pp. 1968–1980, 2016.

[13] A. Karami and M. Guerrero-Zapata, "An ANFIS-based cache replacement method for mitigating cache pollution attacks in named data networking," *Comput. Netw.*, vol. 80, pp. 51–65, 2015.

[14] H. Park, I. Widjaja, and H. Lee, "Detection of cache pollution attacks using randomness checks," in *Proc. IEEE Int. Conf. Commun.*, 2012, pp. 1096–1100.

[15] Z. Xu, B. Chen, N. Wang, Y. Zhang, and Z. Li, "ELDA: Towards efficient and lightweight detection of cache pollution attacks in NDN," in *Proc. IEEE 40th Conf. Local Comput. Netw.*, 2016, pp. 82–90.

[16] M. Conti, P. Gasti, and M. Teoli, "A lightweight mechanism for detection of cache pollution attacks in named data networking," *Comput. Netw.*, vol. 57, no. 16, pp. 3178–3191, 2013.

[17] L. Breslau, P. Cao, L. Fan, G. Phillips, and S. Shenker, "Web caching and zipf-like distributions: Evidence and implications," in *Proc. Int. Conf. Comput. Commun.*, 1999, pp. 126–134.

[18] Y. Gao, L. Deng, A. Kuzmanovic, and Y. Chen, "Internet cache pollution attacks and countermeasures," in *Proc. IEEE Int. Conf. Netw. Protocols*, 2006, pp. 54–64.

[19] L. Deng, Y. Gao, Y. Chen, and A. Kuzmanovic, "Pollution attacks and defenses for internet caching systems," *Comput. Netw. Int. J. Comput. Telecommun. Netw.*, vol. 52, no. 5, pp. 935–956, 2008.

[20] M. Xie, I. Widjaja, and H. Wang, "Enhancing cache robustness for content-centric networking," in *Proc. Int. Conf. Comput. Commun.*, 2012, pp. 2426–2434.

[21] C. Ghali, G. Tsudik, and E. Uzun, "Needle in a haystack: Mitigating content poisoning in named-data networking," in *Proc. Workshop Secur. Emerging Netw. Technol.*, 2014, pp. 68–73.

[22] H. Guo, X. Wang, K. Chang, and Y. Tian, "Exploiting path diversity for thwarting pollution attacks in named data networking," *IEEE Trans. Inf. Forensics Secur.*, vol. 11, no. 9, pp. 2077–2090, May 2017.

[23] H. Salah, M. Alfatafta, S. SayedAhmed, and T. Strufe, "CoMon++: Preventing cache pollution in NDN efficiently and effectively," in *Proc. 42nd IEEE Conf. Local Comput. Netw.*, 2017, pp. 1–9.

[24] G. Zhang, J. Liu, X. Chang, and Z. Chen, "Combining popularity and locality to enhance in-network caching performance and mitigate pollution attacks in content- centric networking," *IEEE Access*, vol. 5, pp. 19012–19022, 2017.

[25] D. Xu and Y. Tian, "A comprehensive survey of clustering algorithms," *Ann. Data Sci.*, vol. 2, no. 2, pp. 165–193, 2015.

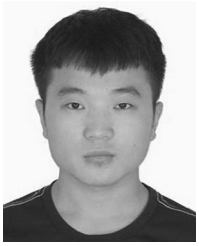
[26] A. Rodriguez and A. Laio, "Clustering by fast search and find of density peaks," *Sci.*, vol. 344, no. 6191, pp. 1492–1496, 2014.

[27] A. Afanasyev, I. Moiseenko, L. Zhang, et al., "ndnSIM: NDN simulator for NS-3," University of California, Los Angeles, Los Angeles, CA, Tech. Rep. 4, 2012.

[28] E. J. Rosensweig, J. Kurose, and D. Towsley, "Approximate models for general cache networks," in *Proc. Int. Conf. Comput. Commun.*, 2010, pp. 1–9.

Lin Yao is an associate professor in School of Software, Dalian University of Technology (DUT) and School of DUT-RU Information Science and Engineering, China. Her research interests include privacy of big data and securty of vanet, CCN, and social network.

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on January 23,2025 at 04:08:39 UTC from IEEE Xplore. Restrictions apply.



Zhenzhen Fan is working toward the ME degree in the School of Software, Dalian University of Technology. His research interests include security and privacy in NDN.

Jing Deng (M'02, SM'13, F'17) is a professor with the Department of Computer Science, University of North Carolina at Greensboro, Greensboro, North Carolina. His research interests include online social networks, wireless networks, and network security. He is a fellow of the IEEE.

Xin Fan is currently a professor of Dalian University of Technology, and Dean of School of DUT-RU Information Science and Engineering. His current research interests include computational geometry and machine learning, and their applications to machine vision and image processing and analysis. He is a senior member of the IEEE.

Guowei Wu is a professor in School of Software, Dalian University of Technology (DUT). His research interests include embedded real-time system, cyber-physical systems (CPS), wireless sensor networks, and NDN.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on January 23,2025 at 04:08:39 UTC from IEEE Xplore. Restrictions apply.